

Algorithm

Complexity Analysis of Divide & Conquer

Solving Recurrence, $T(n) = a T(n/b) + f(n)$

Annotations:

- a : number of sub-problems
- n/b : size of each sub problem
- $f(n)$: cost of dividing
- b : factor for divide the problem

Binary Search,

$$T(n) = T(n/2) + \theta(1)$$

Annotations:

- $n/2$: working with the half part
- $\theta(1)$: dividing $\text{mid} = (\text{left} + \text{right})/2$

3 ways of solving these recurrence:

- ① Substitution Method.
- ② Recurrence Tree Method.
- ③ Master Method.

Substitution Method: (is for proving that the complexity is right)

- * guess the complexity mathematical
- * Use then Induction to prove that the guess is correct.

Assume $T(n) \leq c \log n$ where $n \geq n_0$ and $c > 1$

$$T(n) = T(n/2) + \theta(1) \quad \left| \begin{array}{l} \text{let,} \\ \theta(1) \rightarrow c_2 (\text{constant}) \end{array} \right.$$

$$= c \log n/2 + c_2$$

$$= c \log n - \underbrace{c \log 2}_{\text{constant term}} + c_2$$

$$= c \log n - c + c_2$$

$$= c \log n - (c - c_2)$$

$$T(n) \leq c \log n \quad \text{when } (c \geq c_2)$$

Merge Sort:

$$T(n) = 2T(n/2) + c_2 n \rightarrow \text{in merging part}$$

$O(n) \rightarrow c_2 n$

Assume $T(n) \leq C n \log n$ where $n \geq n_0$ and $C \geq 1$

$$T(n) = 2T(n/2) + c_2 n$$

$$= 2C \frac{n}{2} \log \frac{n}{2} + c_2 n$$

$$= 2C n \log n - 2C n \log 2 + c_2 n$$

$$= n(2C \log n - 2C \log 2 + c_2)$$

$$= n \{ 2C \log n - (2C \log 2 - c_2) \}$$

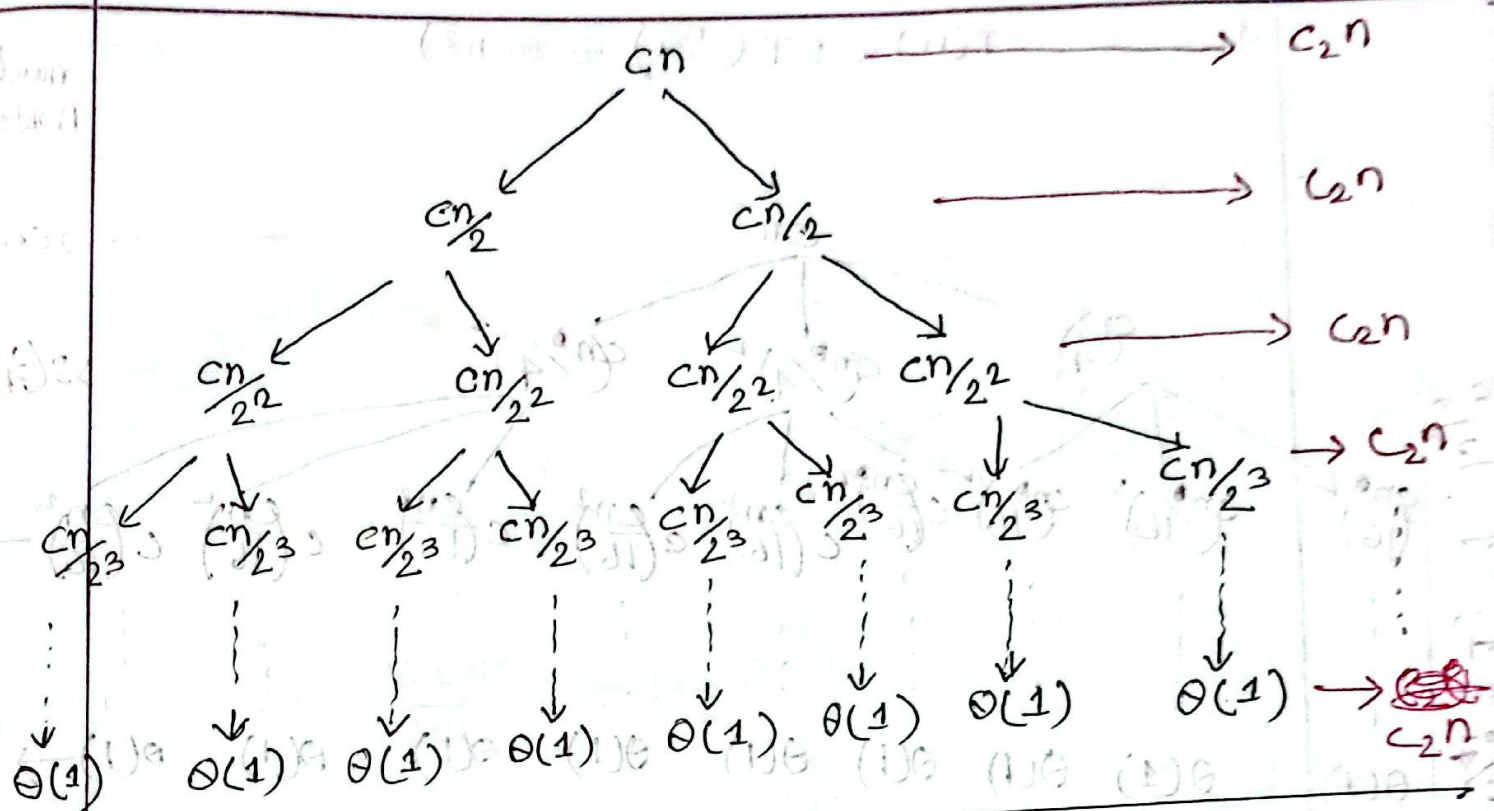
$$\leq n \underbrace{2C \log n}_{\text{constant}}$$

$$T(n) \leq C n \log n$$

Recurrence Tree :

$$T(n) = 2T(n/2) + c_2 n$$

$\sum_{i=0}^{\log n} (\text{depth})^i = \log n$



$$= \underbrace{\log n}_{\text{recurrence depth}} \cdot c_2 n + \boxed{cn} \rightarrow \text{leaves}$$

$$\Rightarrow \log n \cdot c_2 n + cn$$

$$\Rightarrow c_2 n \log n + cn$$

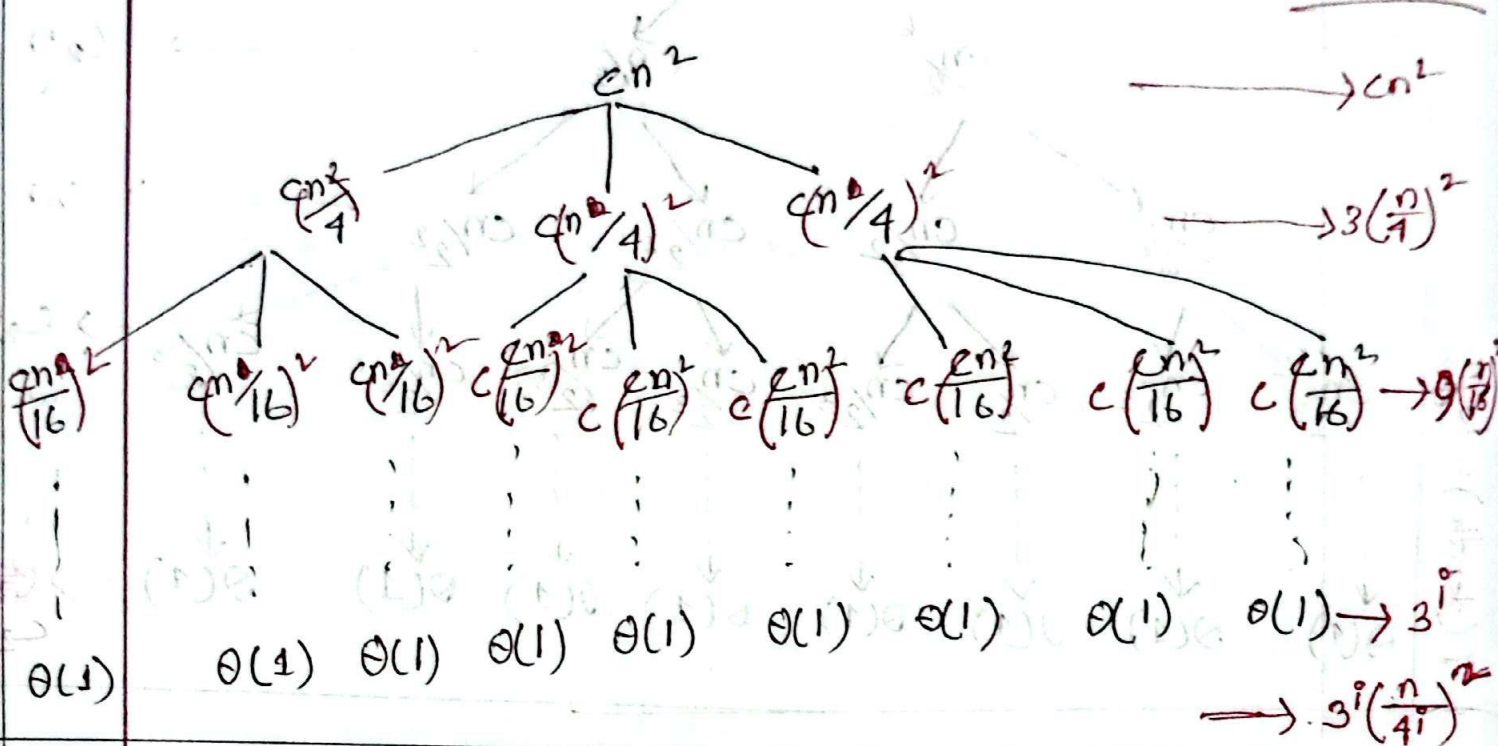
$$\leq c_2 n \log n \quad \text{where } c_2 \text{ large enough than } e$$

$$\sum_{i=0}^{\log n} \left(\frac{e}{e}\right)^i = \sum_{i=0}^{\log n} 1 = \log n + 1$$

$$T(n) = 3T\left(\frac{n}{4}\right) + \theta(n^2)$$

number of Problems

$n/4^i = 1; i = \log_4 n$



$$T(n) = cn^2 + 3c\left(\frac{n}{4}\right)^2 + 3^2c\left(\frac{n}{4^2}\right)^2 + \dots + 3^i c\left(\frac{n}{4^i}\right)^2$$

$$= cn^2 \left(1 + \frac{3}{4} + \frac{3^2}{4^2} + \dots + \frac{3^i}{4^i}\right) + c_2 3^{\log_4 n}$$

$$= cn^2 \left(1 + \frac{3}{4} + \frac{3^2}{4^2} + \dots + \frac{3^i}{4^i}\right) + c_2 3^{\log_4 n}$$

$$= cn^2 \sum_{i=0}^{\log_4 n} \left(\frac{3}{4}\right)^i + c_2 3^{\log_4 n}$$

$$T(n) = cn^2 \sum_{i=0}^{\log_4 n} \left(\frac{3}{16}\right)^i + c_2 3^{\log_4 n}$$

$$\leq cn^2 \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i + c_2 3^{\log_4 n}$$

$$\leq cn^2 \frac{1}{1 - \frac{3}{16}} + c_2 3^{\log_4 n}$$

$$\leq \frac{16cn^2}{13} + c_2 3^{\log_4 n}$$

$$\leq c_3 n^2 + c_2 n^{\log_4 3}$$

where $c_3 = \frac{16c}{13}$

$$\leq c_3 n^2 + c_2 n^{\approx 0.8}$$

$$O(n^2 + n^{0.8})$$

$$= O(n^2)$$

Formula:

If $r < 1$,

$$r + r^2 + r^3 + \dots$$

$$= \left(\frac{1}{1-r}\right)$$

$$a^{\log_b c} = c^{\log_b a}$$

Algorithm

Master, Method

$$T(n) = aT(n/b) + f(n)$$

Case 1:

if $f(n) = O(n^{\log_b a - \epsilon})$ where $\epsilon > 0$, then

$$T(n) = O(n^{\log_b a})$$

in the tree, it represents leaf level complexity

Case 2:

If $f(n) = \Theta(\underbrace{n^{\log_b a}}_{\substack{\text{in the tree, it} \\ \text{represents leaf level complexity}}} \log^k n)$ for $k \geq 0$, then

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

Case 3:

If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for $\epsilon > 0$ and

$a f(n/b) \leq c f(n)$ where $c > 1$ and n sufficiently

larger then $T(n) = \Theta(f(n))$

Example:

$$T(n) = 2T(n/2) + n$$

$$\text{So, } n^{\log_2 2} \log^0 n \Rightarrow n^1 \cdot 1 \Rightarrow n \Rightarrow f(n)$$

Falls under case 2

$$\text{so } T(n) = n^{\log_2 2} \log^{0+1} n \Rightarrow n \log n$$

example 2:

$$T(n) = 3T(n/4) + n^2$$

$$a = 3, b = 4$$

$$\text{so, } n^{\log_4 3} \Rightarrow n^{\log_4 3 + \epsilon} \Rightarrow n^{\log_4 16} \Rightarrow n^2$$

$$f(n/4) = \frac{n^2}{16} \quad f(n) = n^2$$

$$3f(n/4) = \frac{3n^2}{16}$$

$$\leq \left(\frac{1}{2}\right)n^2 \quad c < 1$$

so, falls under case 3:

$$\text{therefore } T(n) = \Theta(f(n)) \Rightarrow \Theta(n^2)$$

Example 3:

$$T(n) = 2T(n/2) + \Theta(1)$$

$$; a = 2, b = 2$$

$$n^{\log_2 2}$$

$$\Rightarrow n^{\log_2 2 + \epsilon} \Rightarrow n^{\log_2 4} \Rightarrow n^2$$

$$n^{\log_2 1}$$

$$= n^0 = 1 = \Theta(1)$$

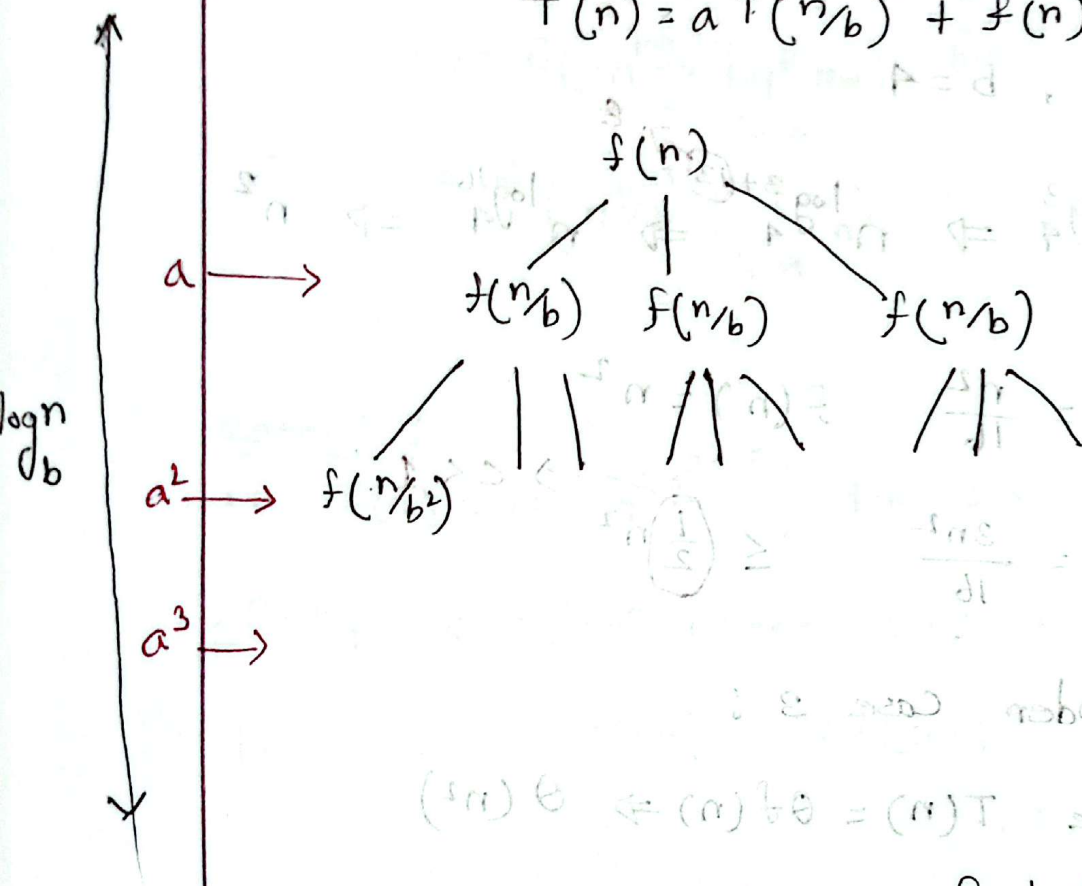
[level base] $(\log n)$

falls under case 1;

$$\text{So, } T(n) = O(n^{\log_2 2})$$

For the cases what can we get using recurrence tree

$$T(n) = aT(n/b) + f(n)$$



in case of depth i .

$$f\left(\frac{n}{b^i}\right)$$

$f(1) \rightarrow \frac{n}{b^i} \approx 1$ (ii) $i = \log_b n$

in the leaf nodes there will be a^i subproblems.
 $a^i \Rightarrow a^{\log_b n} \Rightarrow n^{\log_b a} \rightarrow$ these number of $O(1)$ problem
 $\therefore \Rightarrow O(n^{\log_b a})$ [leaf level]

each level complexity $\rightarrow f(n)$

To get to the i th depth total complexity

$$i = \log_b^n * f(n) ?$$

$$\therefore \text{Total complexity} = O(n^{\log_b a}) + \log_b^n * f(n)$$

In case 2, $n^{\log_b a}$ dominates the breaking part.
And represents that each level needs same amount of complexity.

Algot Algorithm

Greedy Algorithm

Feasible Solution :

subset of the problem that satisfy certain conditions fixed by the problem.

Objective Function :

Tries to maximize or minimize a function's value based on the feasible solutions.

Optimal solution :

Among all the feasible solution those solutions that maximize or minimize the objective function.

subset Paradigm :

ordering paradigm :

OS $\leftarrow (0, 0, 1)$

OS $\leftarrow (1, 0, 0)$

OS $\leftarrow (0, 1, 0)$

Fractional knapsack

n objects, A knapsack bag with capacity m
weight of n objects $w = \{w_1, w_2, w_3, w_4, \dots, w_n\}$

profit of n objects $p = \{p_1, p_2, p_3, \dots, p_n\}$

Objectives maximize $\sum_{i \in \text{objects}} x_i * p_i$ $\left\{ \begin{array}{l} 0 \leq x_i \leq 1 \end{array} \right.$

feasibility $\rightarrow \sum_{i \in \text{objects}} x_i * w_i \leq m$

$n = 3$ $m = 20$ $p = \{25, 24, 15\}$

$w = 9, p = 12.5$
 $w = 5, p = 8$
 $w = 2.5, p = 3.75$
 $w = 3, p = 7.5$
 $W = \{18, 15, 10\}$
 $\xrightarrow{\text{total weight}} 16.5 \xrightarrow{\text{total profit}} 24.25$

$\rightarrow (1, 2/15, 0) \rightarrow 20 \rightarrow 28.2$

$\rightarrow (0, 2/3, 1) \rightarrow 20 \rightarrow 31$

$\rightarrow (0, 1, 1/2) \rightarrow 20 \rightarrow 31.5$

subset
paradigm

SRS

Backlog Refinement

Scope creep $\rightarrow$ ~~less~~ ^{more} ~~expand~~ ^{expand}

Algorithm

$n = 4$

$$P = \{100, 10, 15, 17\} \quad d = \{2, 1, 2, 1\}$$

n Jobs with deadline for each d_i and profit P_i
each jobs needs 1 unit of time to finish
what is the maximum profit can be gained
without overlapping any jobs.

Jobs	seq	profit
$\{1, 2\}$	2, 1	110
$\{1, 4\}$	4, 1	127
$\{1, 3\}$	1, 3 or 3, 1	15
$\vdots$	$\vdots$	$\vdots$

Optimal $\{1, 4\}$ with sequence $\{4, 1\}$ profit 127



going to take the most profit at the end of the deadline $\rightarrow$ index. then take profit decrease decreasing order and put them before the highest profit.

Algorithm JobSchedule(J, d, P, n) {

sort(J) descending based on profit
profit := 0;

for $i = 1$ to n do

for $s := d[i]$ to 1 do

if slot $[s]$ is empty then

assign slot $[s]$ to job $J[i]$

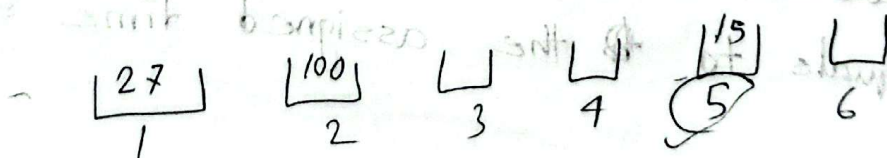
profit := profit + $P[i]$;

break;

return profit;

}

$P = \{100, 27, 15\}$, $d = \{2, 1, 5\}$



Theorem :

J is a set of k jobs and
 $\sigma = i_1, i_2, i_3, i_4, \dots, i_k$ is a permutation of 1 to k ,

such that $d_{i_1} \leq d_{i_2} \leq \dots < d_{i_k}$. Now J is a feasible solution iff jobs in J can be processed with the sequence σ without any overlaps.

$$d = \{ 7 \quad 8 \quad 9 \quad 12 \quad 11 \quad 10 \}$$

$$\text{jobs} = \{ 1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6 \}$$

$$7 \leq 8 \leq 9 \leq 10 \leq 11 \leq 12$$

$$\sigma = \{ 1 \quad 2 \quad 3 \quad (6) \quad 5 \quad 4 \}$$

$$q = 1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6$$

take it as a time slot

Proof

J is feasible for sequence $\sigma' = r_1, r_2, \dots, r_k$ if
 $d_{r_q} \geq q$ where $1 < q < k$.

(the deadline of the job should be greater than or equals to the assigned time slot)

Assume $\sigma' \neq \sigma$

The least index a where $i_a \neq p_a$
 $p_b \neq i_a, b > a$

$$\begin{cases} \sigma' = p_1, p_2, p_3, \dots, p_b, \dots, p_k \\ \sigma = i_1, i_2, i_3, \dots, i_c, \dots, i_k \end{cases}$$

Here the value of $a = 3$

~~The least index~~

$$\begin{matrix} dr_3 > dr_b \\ \downarrow & \downarrow \\ d_{i_c} & d_{i_3} \end{matrix}, c > 3$$

if, $dra \geq dr_b$

Swap (p_b, p_a)

So, we can say,

$$dr_b \geq dra$$

$$as, \quad dr_b \geq b \quad b > a$$

$$dra \geq a \quad dr_b \neq dra$$

$\Rightarrow$ not always.